

Lógica Fuzzy na Classificação de Anomalias em APIs REST

Fuzzing guiado por contrato + Sistema de Inferência Fuzzy

Levi Gomes - Luiz Garcia - Natã Cezer Bordignon - Renan Balestrin Bez

Contexto e problema

APIs REST são base de microsserviços, apps móveis e integrações B2B. Para testar robustez, o fuzzer usa o contrato OpenAPI e envia entradas mutantes.

O desafio é o **oráculo de teste**: decidir se uma resposta inesperada é uma falha real ou um comportamento aceitável sob estresse.

Abordagens binárias tratam achados muito diferentes como iguais:

- 400 não documentado
- 200 aceitando payload inválido
- 500 por exceção interna

Pergunta e objetivo

Pergunta de pesquisa: um Sistema de Inferência Fuzzy consegue classificar anomalias de APIs REST com mais granularidade que uma abordagem booleana?

O sistema combina três sinais:

- desvio do status code
- tempo de resposta
- razão do tamanho do payload

E gera um **score de criticidade entre 0.0 e 1.0**, depois convertido em **LOW**, **MEDIUM** ou **HIGH**.

Por que Lógica Fuzzy?

A lógica booleana trabalha com fronteiras rígidas: verdadeiro ou falso.

A **Lógica Fuzzy** permite graus de pertinência:

- uma resposta pode ser parcialmente "normal" e parcialmente "lenta"
- um payload pode estar entre "esperado" e "grande"
- um desvio pode ser baixo, médio ou alto

Isso combina com testes de rede, onde latência, tamanho de resposta e comportamento sob estresse não possuem limites perfeitamente fixos.

Variáveis e fluxo do FIS

Variável	Conjuntos
status_deviation	low, medium, high
response_time	normal, slow, timeout
payload_ratio	expected, large, empty
anomaly_score	low, medium, high

OpenAPI

- > payloads mutantes
- > resposta da API
- > tipo da anomalia
- > score fuzzy
- > severidade

O FIS atua como a **camada de inteligência** depois da coleta dos resultados.

Regras de inferência

As regras traduzem conhecimento heurístico em decisões automatizadas:

- SE desvio de status é alto OU tempo é timeout, ENTÃO criticidade é alta
- SE desvio é médio E tempo é lento, ENTÃO criticidade é média
- SE desvio é baixo E payload é grande, ENTÃO criticidade é média
- SE desvio é baixo E tempo é normal E payload é esperado, ENTÃO criticidade é baixa

Depois da inferência, o resultado é desfuzzificado pelo método do **centroide**.

Metodologia: status e mutações

Desvio de status

Situação	Desvio
Status documentado	0.0
Sem referência clara	0.5
4xx não documentado	0.6
5xx ou sem resposta	1.0

Mutações

- string vazia
- string muito longa
- tipo incorreto
- campo removido
- valor numérico fora do limite
- combinações simples

Metodologia: auth, classificação e ambiente

O sistema também testa endpoints protegidos:

- descoberta de registro e login
- extração de JWT
- probes sem token e com token inválido

Tipos técnicos: `SERVER_ERROR`, `UNEXPECTED_SUCCESS`, `UNEXPECTED_ERROR_STATUS`, `ERROR_CONTRADICTION`.

Item	Configuração
Linguagem	Python 3.13
Biblioteca	scikit-fuzzy
API alvo	crAPI
Endpoints	cerca de 60
Fuzzing	Level 1
Limite	até 10 mutações por endpoint

Resultado geral

207

anomalias identificadas

42

classificadas como alta severidade

Severidade	Quantidade
HIGH	42
MEDIUM	120
LOW	45

Somente **20,3%** dos achados foram priorizados como críticos.

Tipos de anomalia encontrados

Classificação	Quantidade	Percentual
UNEXPECTED_ERROR_STATUS	103	49,8%
SERVER_ERROR	42	20,3%
UNEXPECTED_SUCCESS	28	13,5%
ERROR_CONTRADICTION	22	10,6%
EXPECTED_FAILURE	10	4,8%
SUCCESS_CONTRADICTION	2	1,0%

A maioria foi documentação incompleta de erros 400 ; os casos mais graves foram erros 500 e 503 .

Casos representativos

Caso	Exemplo	Score	Severidade
Erro de servidor	<code>/verify-email-token</code> retornou <code>500</code>	0.84	HIGH
Erro não documentado	login retornou <code>400</code> fora do contrato	0.60	MEDIUM
Sucesso inesperado	<code>/reset-password</code> aceitou payload inválido	0.50	MEDIUM
Rejeição correta	erro <code>400</code> documentado	0.31	LOW

O FIS diferencia falhas graves, drift de contrato e casos de baixa prioridade.

Discussão

Pontos fortes

- priorização clara das falhas críticas
- score contínuo em vez de rótulo binário
- combinação de contrato, desempenho e payload
- potencial para execução em CI/CD

Limitações

- muitos 400 ficaram em severidade média
- thresholds podem ser ajustados
- regras ainda são estáticas, sem feedback adaptativo

Conclusão e próximos passos

A Lógica Fuzzy mostrou-se adequada para tratar a incerteza nos testes de robustez de APIs REST.

Comparada a um oráculo binário, a abordagem reduziu a prioridade imediata de **207 achados** para **42 anomalias HIGH**.

Próximos passos:

- ajustar funções de pertinência e thresholds
- adicionar feedback do desenvolvedor
- detectar stack traces no corpo da resposta
- integrar o score fuzzy a pipelines de CI/CD